

Software Testing.

Testing is a process of executing a program with the intent of finding errors.

Its types are:

- **Functional testing:** Testing done to ensure the every functions work efficiently or not. Eg: clicking product description navigates user to next page or not.
 - Unit testing: Test done individually before merging the units.
 - Integration testing: Test done as a whole. Units are integrated all together. There are many forms of integration testing like:
 - Bigbang: tests everything at once
 - Top-down: tests from top to bottom
 - Bottom-up: tests from down to up
 - Sandwich: combination of top-down and bottom-up
 - Acceptance testing : test done from the end-user's perspective.
 - Smoke testing: It tests core functionality. It checks critical functions of a system like login, logout, save, etc.
 - Sanity testing: It tests the new functionalities work correctly or not. It ensures the new changes are correct.
- **Non-functional testing:** Testing done outside the functionalities of a system. Eg: testing the system is responsive, security is good or bad, etc.
 - Performance testing: software is tested to ensure how good it handles the traffic load and datas.
 - Compatibility testing: software is testing to ensure it is compatible or not. Eg: is it supported by different browsers and devices over different network and different regions.

- Security testing: done to check how good the security is, how it can be prevented from malicious programs.
- Visual testing: checks UI rendering and all kinds of layout.

Testing Techniques Types:

- **Static testing:** Testing done without running the program. It checks software's documentation before the execution.
- **Dynamic testing:** It tests software during the operation. It is an ongoing technique.

Software Testing Techniques:

- **Equivalence Class Partitioning (ECP):** It is a black box testing technique. It divides the task into valid and invalid sections. It tests one value per class. If one test in a partition works, the other also behaves similarly. This reduces the number of test cases but also covers maximum functionality.

Eg: The accepted age range is 1 - 100. So, valid age of a person is 1 - 100 and the invalid age range is below 1 and above 100.

Eg: In mobile banking app, the transaction limit is 100 - 10000. So test 100 is valid but test 50 is invalid.

- **Boundary Value Analysis (BVA):** It is also a black box testing technique. It usually tests maximum and minimum values or just below and just above values. It tests the limits of ranges where errors are more likely to occur.

Eg: Age range is 10, 50, so it tests 9, 10, 11, 49, 50, 51.

Eg: In mobile banking app, the amount must be at least 100. So, test 1 is 99, test 2 is 100, test 3 is 101.

- **Decision Table Testing:** It tests the system by associating inputs and its corresponding outputs in a structured table format. It uses a table of conditions vs actions to calculate output. It ensures complete coverage of input test cases. It doesn't overlook any aspect regarding the test cases.

Eg: A receipt has two different segments; product name and price. So, it tests those two segments to ensure the correct outcome.

Eg: Conditions (Student got good grades (Y/N), Student got good attendance (Y/N))

Actions (Apply 50% scholarship, No scholarship)

In this scenario, if test 1 have both the conditions yes, then scholarship is given. If test 2 has only one of the conditions yes, then the scholarship is not given.

- **Cause-effect graphing:** It model input conditions. It uses graph to represent the cause and effect of a test case. Testers link up input and expected output.

Eg: In a scholarship approval, attendance and academics is examined to give the scholarship.

In this scenario, in order to get that result (scholarship approval), both cases i.e. attendance and approval (input 1 and 2) should be true.

- **State Transition Testing:** It is performed to check the change of the state in the application as the name suggests. This is done based on the events and inputs. It is useful when the system depends on the events.

Eg: In a security system, there are states (login, logout, account locked) and inputs (correct password, incorrect password). If it's correct password, the state transits from logout to login whereas, if it is incorrect password, the state transits from logout to account locked. Hence, state transitions tests these cases based on input and events.

- **Risk Coverage:** This technique gives priority to the most sensitive and important parts of the system that has more risk of failure. It mostly focuses on risk and damage control. Eg: payment processing is given the priority in mobile banking.
- **Statement Coverage:** It is a white box testing technique. This covers every line single of code. It ensures every code is executed at least once during the testing.

Manual and Automated Testing

Manual testing is executed by human effort without any tools or automation. It is done to where automated testing is not enough or cannot be used. It is low cost compared to automated testing although it consumes time. It is flexible and gives fast feedback.

Scenarios to perform manual testing:

- Exploratory testing: done to explore the new features in a system.
- UI testing: done to check the colors and site looks across different browsers.
- Usability testing: checks if the UI is easy to use.

Automated testing is done totally with tools and automation techniques. It does not require human effort one should be very good in programming and automation skills to do this. It is

costly and is preferred for complex and big projects. It is fast compared to manual testing.

This testing is good for repetitive tests.

Black Box and White Box Testing

Black box testing tests outer parts of code. Testers perform this testing rather than developers. It does not focus on functionality. Mostly manual tools are used. Testers does not need the full knowledge of code.

White box testing tests internal structure of code. It focuses on code logic. One should be fully familiar with the code to perform this so a developer performs this. It is complex and hard to maintain. It reduces overhead communication between testers and developers.